

668d08a

[wpt](#) / [webauthn](#) / [createcredential-resident-key.https.html](#)

chromium-wpt-export-bot and **kreibhauer** 6 years ago

178 lines (174 lines)

Code

Blame

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>navigator.credentials.create() test with residentKey and credProps<
6 <meta name="timeout" content="long">
7 <script src="/resources/testharness.js"></script>
8 <script src="/resources/testharnessreport.js"></script>
9 <script src="/resources/testdriver.js"></script>
10 <script src="/resources/testdriver-vendor.js"></script>
11 <script src=helpers.js></script>
12 <script>
13
14 "use strict";
15 const credPropsTests = [
16   {
17     name: "U2F",
18     authenticatorArgs: {
19       protocol: "ctap1/u2f",
20     },
21     expected: {
22       discouraged: {
23         success: true,
24         hasRk: true,
25         rk: false,
26       },
```

```
27     preferred: {
28         success: true,
29         hasRk: true,
30         rk: false,
31     },
32     required: {
33         success: false,
34     },
35 },
36 },
37 {
38     name: "CTAP 2.0 without resident key support",
39     authenticatorArgs: {
40         protocol: "ctap2",
41         hasResidentKey: false,
42         hasUserVerification: true,
43         isUserVerified: true,
44     },
45     expected: {
46         discouraged: {
47             success: true,
48             hasRk: true,
49             rk: false,
50         },
51         preferred: {
52             success: true,
53             hasRk: true,
54             rk: false,
55         },
56         required: {
57             success: false,
58         },
59     },
60 },
61 {
62     name: "CTAP 2.0 with resident key support",
63     authenticatorArgs: {
64         protocol: "ctap2",
65         hasResidentKey: true,
66         hasUserVerification: true,
67         isUserVerified: true,
68     },
69     expected: {
70         discouraged: {
```

```

71         success: true,
72         // CTAP2.0 authenticators may treat all credentials as discoverable
73         // thus Chrome omits 'rk' in this case.
74         hasRk: false,
75     },
76     preferred: {
77         success: true,
78         hasRk: true,
79         rk: true,
80     },
81     required: {
82         success: true,
83         hasRk: true,
84         rk: true,
85     },
86 },
87 },
88 {
89     name: "CTAP 2.1 without resident key support",
90     authenticatorArgs: {
91         protocol: "ctap2_1",
92         hasResidentKey: false,
93         hasUserVerification: true,
94         isUserVerified: true,
95     },
96     expected: {
97         discouraged: {
98             success: true,
99             hasRk: true,
100             rk: false,
101         },
102         preferred: {
103             success: true,
104             hasRk: true,
105             rk: false,
106         },
107         required: {
108             success: false,
109         },
110     },
111 },
112 {
113     name: "CTAP 2.1 with resident key support",
114     authenticatorArgs: {

```

```

115     protocol: "ctap2_1",
116     hasResidentKey: true,
117     hasUserVerification: true,
118     isUserVerified: true,
119   },
120   expected: {
121     discouraged: {
122       success: true,
123       hasRk: true,
124       rk: false,
125     },
126     preferred: {
127       success: true,
128       hasRk: true,
129       rk: true,
130     },
131     required: {
132       success: true,
133       hasRk: true,
134       rk: true,
135     },
136   },
137 },
138 ];
139
140 for (const fixture of credPropsTests) {
141   for (const rkRequirement of ["discouraged", "preferred", "required"]) {
142     virtualAuthenticatorPromiseTest(async t => {
143       const promise = createCredential({
144         options: {
145           publicKey: {
146             authenticatorSelection: {
147               residentKey: rkRequirement,
148             },
149             extensions: {
150               credProps: true,
151             },
152           },
153         },
154       });
155
156       assert_true(rkRequirement in fixture.expected);
157       const expected = fixture.expected[rkRequirement];
158       assert_true('success' in expected);
159       if (expected.success) {

```

```
159         if (!expected.success) {
160             return promise_rejects_dom(t, "NotAllowedError", promise);
161         }
162
163         const cred = await promise;
164         assert_true('credProps' in cred.getClientExtensionResults());
165         const credProps = cred.getClientExtensionResults().credProps;
166         assert_equals('rk' in credProps, expected.hasRk, "hasRk");
167         if (expected.hasRk) {
168             assert_equals(credProps.rk, expected.rk, "rk");
169         }
170     }, fixture.authenticatorArgs, fixture.name
171         + ": navigator.credentials.create() with credProps extension, rk="
172         + rkRequirement);
173     }
174 }
175 </script>
176 </head>
177 <body></body>
178 </html>
```